

Монгол улсын их сургууль
Мэдээллийн технологи, электроникийн сургууль
Мэдээлэл , компьютерийн ухааны тэнхим

Семинарын даалгавар №4
Arc42 & C4

Гүйцэтгэсэн:

Програм хангамж-4

Э.Түвшин-Эрдэнэ (23b1num0869)

Улаанбаатар хот

2026 он

Arc 42

1 Танилцуулга ба зорилгууд

1.1 Шаардлагын хураангуй

Энэхүү систем нь компьютерийн шинжлэх ухааны ойлголтуудыг тоглоомны арга барилаар заах зориулалттай сургалтын видео тоглоом юм. Энэхүү тоглоомд төгсгөлөг төвийн машин (Finite State Machine)- аар удирдуулах тоглогчийн дүр, интерактив үе үүсгэгч (Level Editor for User-Generated Levels), өөрийн үүсгэсэн үеээ олон нийтэд хуваалцах онцлог ба real-time debugger багаж байх болно.

Системийн зорилгууд:

эрэмбэ	**зорилго**
1	Finite state machine - ий ойлголтыг практик хэлбэрээр тоглогчдод олгох.
2	Тоглогчдод өөрсдийн бүтээсэн үеийг хуваалцах боломж олгох.
3	Хэрэглэгчийн бүтээсэн үеүүдийг аюулгүй, найдвартай байдлаар хүргэх online infrastructure-ийг бүтээн, тогтмол засвар үйлчилгээ хийх.
4	Засан сайжруулах боломжтой байдлаар тоглоомыг хөгжүүлэх.

Гол функциональ шаардлагуудын хураангуй:

ID	**Шаардлага**	**Тайлбар**
1	Тоглогчийн гол дүрийн FSM хөдөлгөөн	Систем нь тоглогчийн дүрийн хөдөлгөөнийг Finite State Machine (Idle, Walking, Running, Jumping, Action) төлөвүүдийн дагуу гүйцэтгэж, товчлуурын оролтоор саадгүй шилжүүлнэ.
2	Интерактив үе редактор	Систем нь тоглогчдод драг-энд-дроп (drag-and-drop) болон FSM ашиглан шинэ үе болон дүрийн логик бүтээх визуаль орчныг олгоно.

3	Өөрийн бүтээсэн үеийг нийтлэх	Тоглогч өөрийн бүтээсэн үеийн шалгуур тестийг (Solvability test) даван туулсны дараа олон нийтийн сүлжээнд нийтлэх боломжтой.
4	Бусад тоглогчдын бүтээсэн үеийг тоглох	Тоглогч олон нийтийн сангуудаас бусад хэрэглэгчдийн үүсгэсэн үеүүдийг хайх, шүүх, тоглох боломжтой.

1.2 Чанарын зорилгууд

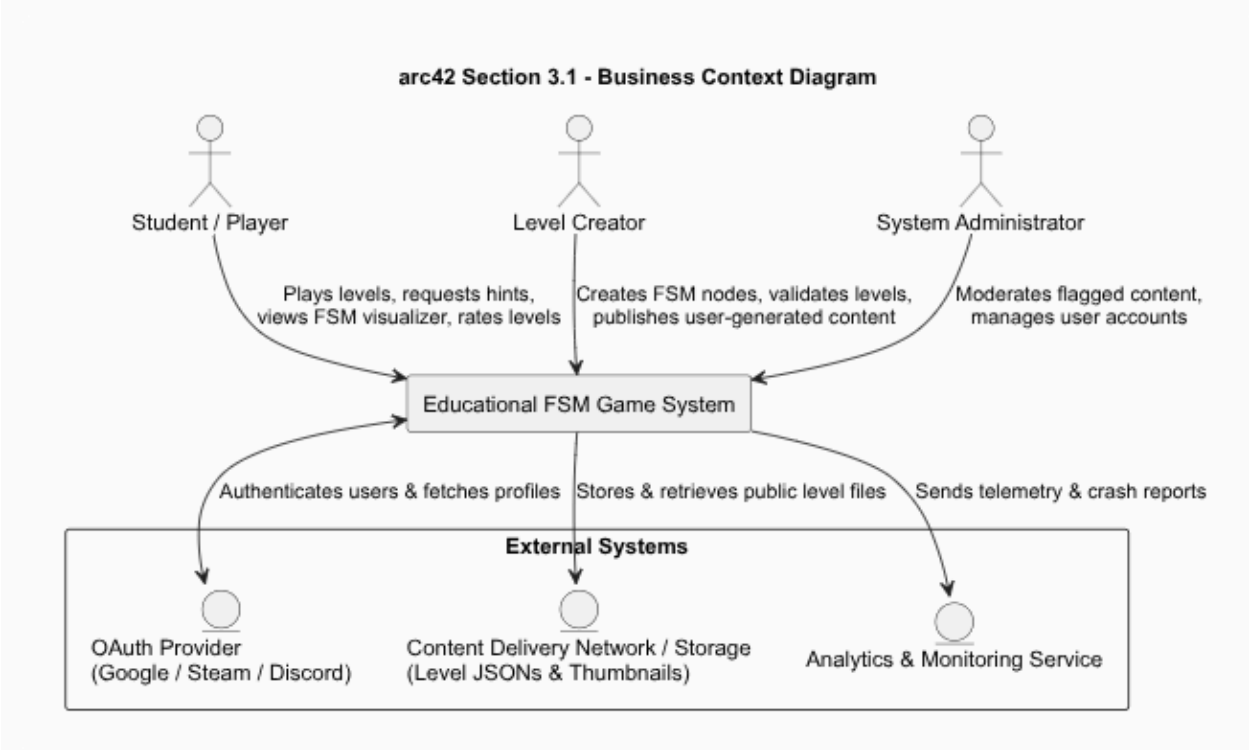
Чанарын зорилго	**Гол зорилго**	**Хэмжүүр**
Гүйцэтгэл (Performance)	Хөдөлгөөнийг саадгүй байлгахын тулд фреймийн хоцрогдол бага байх ба үеүүдийг хурдан ачааллуулах.	FSM state transition latency $< 16.6ms$ (p95, 60 FPS). Community level load time $< 1.5s$ (95th percentile).
Аюулгүй байдал(Security)	Тоглогчдийн бүтээсэн үеүүдийн файл хор хөнөөлтэй байхаас сэргийлэх.	100% server-side JSON payload sanitization against XSS/RCE. Verified via OWASP ZAP.
Найдвартай байдал(Reliability)	Серверийн нээлттэй байдал өндөр ба уналтаас хурдан сэргэх.	Server uptime $\ge 99.9\%$. Local FSM state auto-recovery duration $< 500ms$.
Арчлан сайжруулахуйц байдал (Maintainability)	Өргөжүүлж болохуйц FSM архитектур.	Adding new states/triggers requires changing $< 10\%$ of core engine code (SonarQube analysis).
Хүртээмжтэй байдал(Accessibility)	Харааны бэрхшээлтэй хүмүүст хүртээлтэй UI/UX сонголт.	WCAG 2.1 AA compliant ($> 4.5:1$ contrast) with Deuteranopia/Protanopia colorblind filters.

1.3 Оролцогч талууд

Role	Contact/ Target Audience	Goal/expectation
Сурагч, тоглогч	Primary users	Сонирхолтой тоглоом, ойлгогдохуйц FSM логик, найдвартай удирдлага (responsive controls)
Үе бүтээгч	Power users	Алдаагүй, ашиглахад хялбар үе бүтээгч
Багш, сургагч	Academic Stakeholders	Компьютерийн ухааны ойлголтыг тодорхой байдлаар харуулах.
Системийн админ	Operations Team	Content moderation багаж хэрэгсэл

3 Хамрах хүрээ

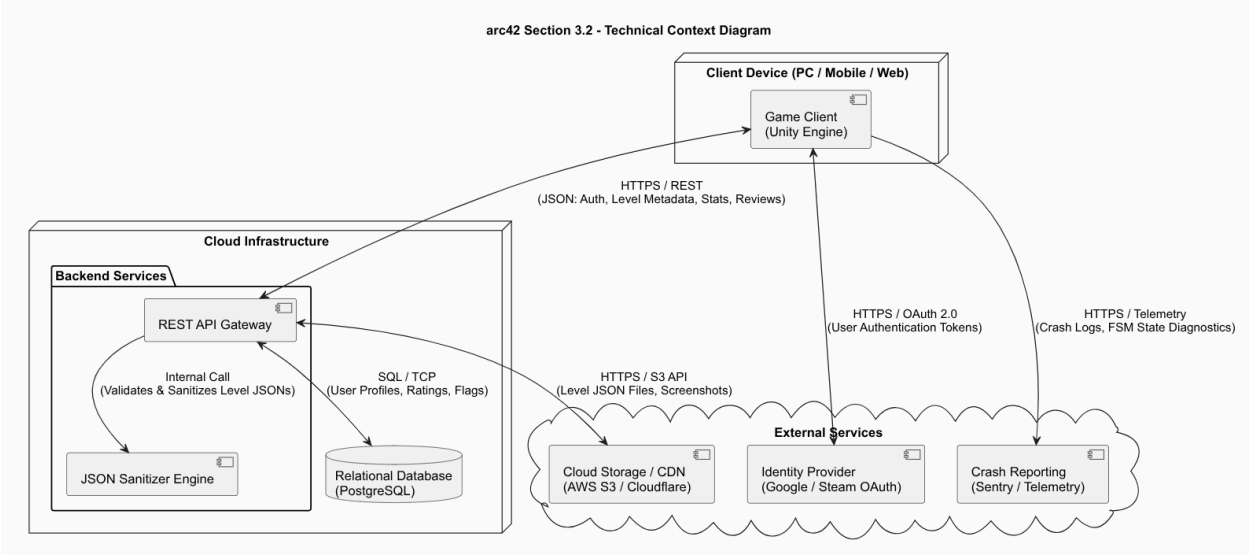
3.1 Бизнесийн хамрах хүрээ



Харилцааны интерфейсүүд:

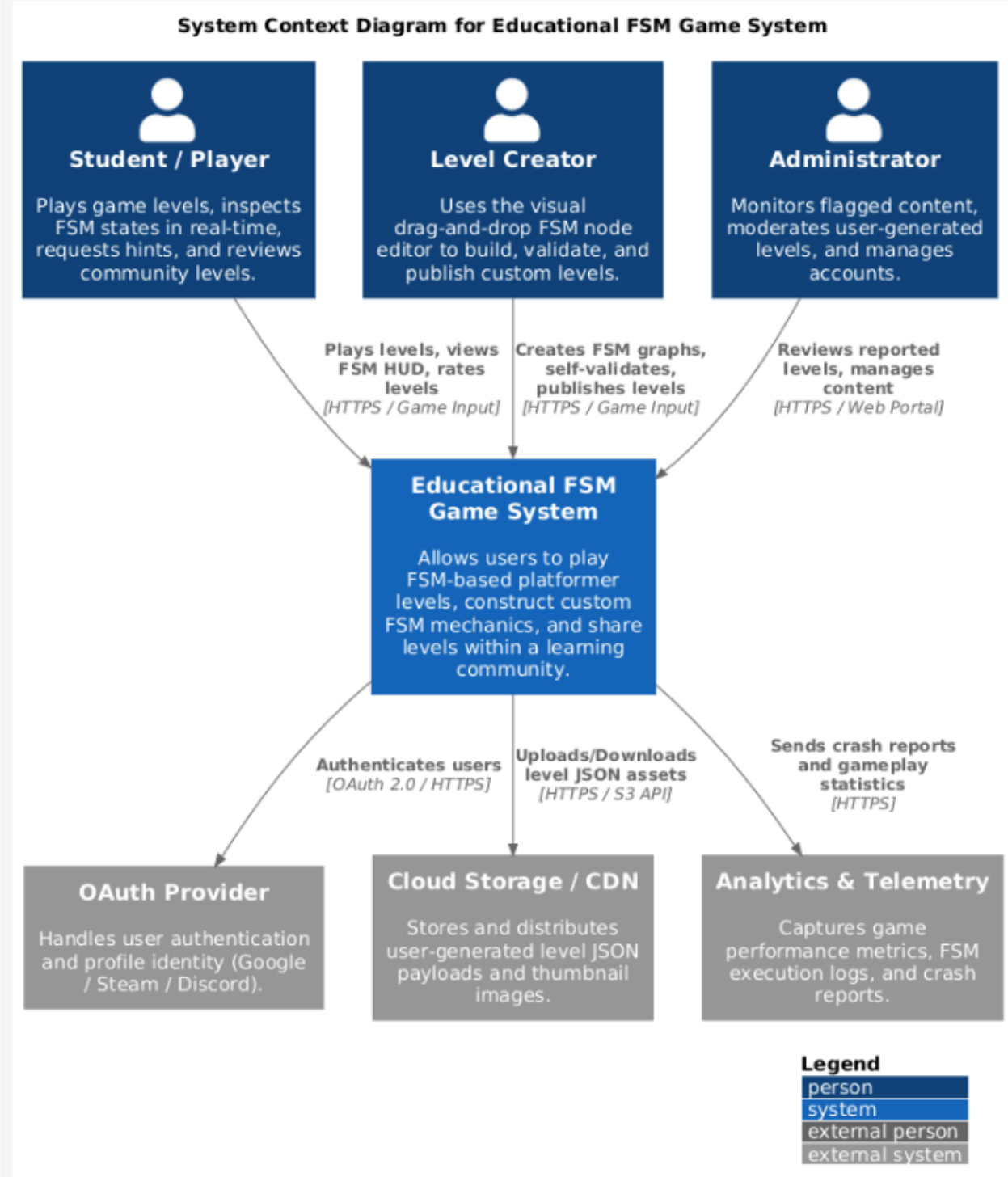
Neighbor / Actor	**Input to System**	**Output from system**	**protocol/channel**
player/creator	Player input (WASD, Space), Node creation, level publishing, hint requests, reviews	Rendered game state, Visual FSM HUD, Level list, Statistics, Hints	Game Engine Input System/ HTTPS REST API
OAuth providor	Auth tokens, User profile identity	Authentication requests, Scope claims	OAuth 2.0 / HTTPS
CDN / Blob Storage	Uploaded level JSON definitions, Thumbnails	Cached level assets, Metadata	HTTPS / S3-compatible API
Administrator	Content removal actions, Moderation rules	Reported level queues, Flag logs	HTTPS Admin Portal

3.2 Техникийн хамрах хүрээ

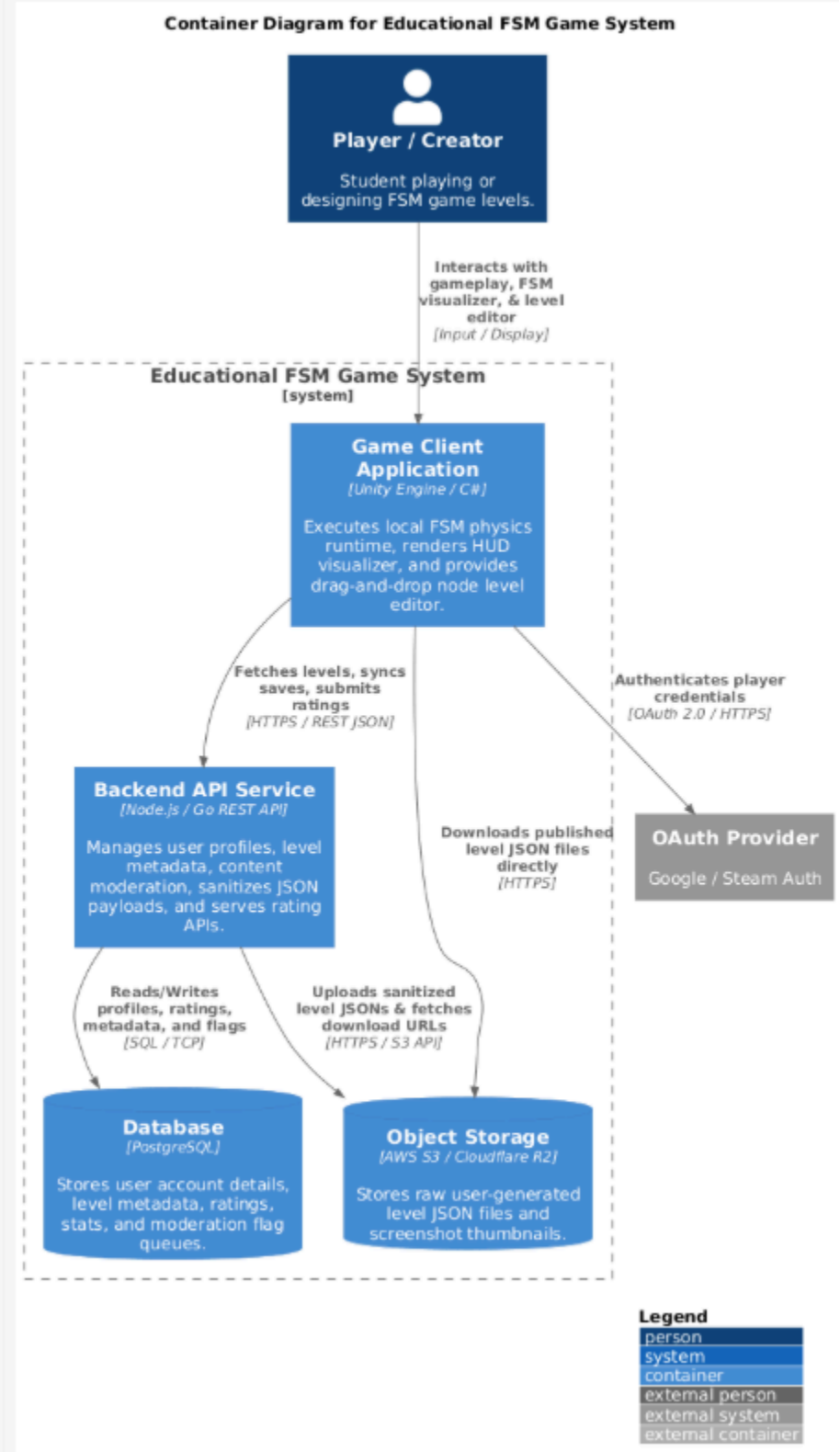


5 Building Block View

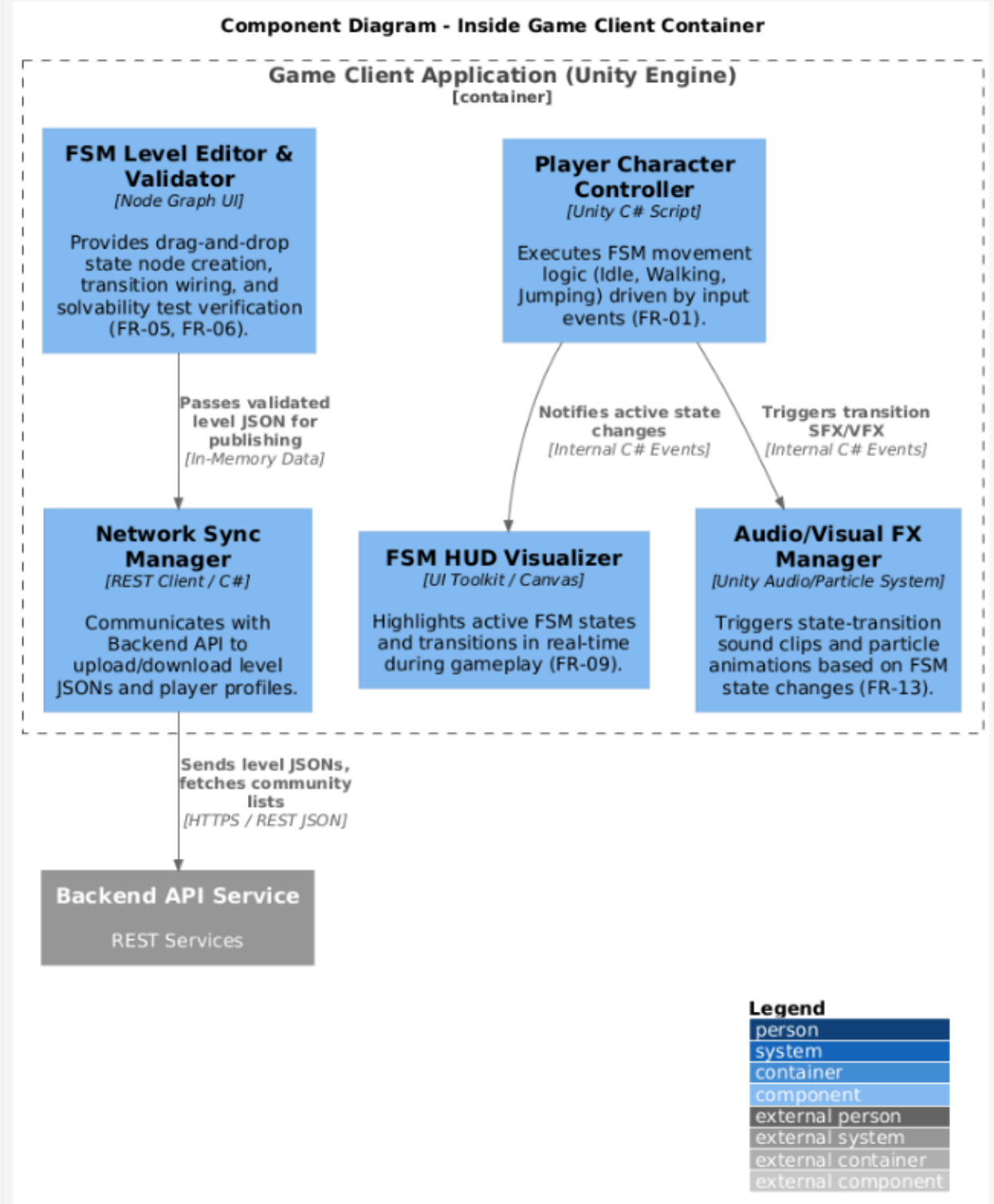
5.1 Level 1 C4 System context



5.2 Level 2 C4 Container



5.2 Level 3 C4 Component



Architectural Decision Records

ADR1: Game engine, client tech stack

- Status: Accepted
- Decider(s): Game architect/ lead developer
- Date: 2026-09-28

Context and Problem statement

Программчлалын ойлголтыг сурч байгаа сурагчид шуурхай, ойлгомжтой feedback байхгүй бол хийсвэр төлвийн логикийг сурахад хүндрэлтэй байдаг. Энэ асуудлыг шийдэхийн тулд (persona pain point) клиент тал high-frame-rate physics болон тогтвортой FSM шилжилтүүдийг(NFR-01: $< 16.6ms$ latency) хийдэг байх хэрэгтэй.

Decision Drivers

- **Persona Pain Point**: “As a student learner, abstract state diagrams in textbooks are hard to map to actual program execution”
- **Persona Pain Point**: “As a student creator, laggy or clunky node editors make building custom levels frustrating”
- **Technical Constraints**: Must support 60 fps gameplay, cross-platform compilation, and interactive 2D canvas UI for node editing

Considered Options

1. Unity Engine (C#)
2. Godot Engine (GDScript/ C#)
3. Web-based Engine (Phaser.js/ html5 canvas)

Decision Outcome

Chosen Option: Unity Engine (C#)

Positive Consequences

- **Robust UI Toolkit & GraphView** : Unity provides built-in node-graph primitives, making the drag-and-drop FSM level editor easier to build and optimize
- **High Performance Engine** : Guarantees p95 frame latency < 16.6ms (60 FPS) for physics and FSM transitions.
- **Strong C# Type System**: Object-oriented structure makes applying the State Pattern clean, modular, and maintainable.

Negative Consequences

- Larger build executable size compared to web frameworks
- Requires WebGL assembly optimization if targeted for web browsers.

ADR2: Level Asset & Data Persistence Strategy

- Status: Accepted
- Decider(s): Lead architect & backend developer
- Date: 2026-09-28

Context and Problem statement

The application relies heavily on User-Generated Content (UGC), allowing students to build custom FSM levels, publish them, and play levels made by others. We need a persistence strategy that handles both structured relational metadata (users, ratings, stats, flags) and variable-length node-graph level definitions safely and rapidly.

Decision Drivers

- **Persona Pain Point**: “As a student lplayer, loading community levels takes too long, ruining the game flow”
- **Persona Pain Point**: “As a player, i worry that downloading community levels might run malicious code or crash my game”
- **Technical Constraints**: Community level load times must be <1.5s, JSON payloads must be sanitized against XSS/RCE, and database uptime must be >= 99.9%.

Considered Options

- Hybrid approach: PostgreSQL + AWS S3/CDN (Level JSON File Storage)
- Pure monolithic RDBMS: Store everything inside postgresql.
- Pure nosql document database: store users stats and level JSONs in MongoDB.

Decision Outcome

Chosen Option: Option1: Hybrid Approach (PostgreSQL + S3/CDN Storage)

Positive Consequences

- Fast Level Fetching: Serving static JSON level files directly from CDN edge caching achieves <1.5s load times globally.
- Security & Isolation: Uploaded level JSONs pass through a dedicated server-side sanitization service before writing to S3, preventing execution of malformed payloads.
- ACID Compliance for metadata: PostgreSQL handles user profiles, ratings, moderation flags, and creator stats with strong relational integrity.

Negative Consequences

- Dual-storage architecture requires synchronization handling(if a level metadata record is deleted, its S3 file must also be purged)

ADR3: System interface & communication protocol

- Status: Accepted
- Decider(s): System architect
- Date: 2026-09-28

Context and Problem statement

The Unity client needs to exchange data with backend services for level browsing, publishing, statistics synchronization, and peer support hints. We need and interface design that balances lightweight payload delivery, implementation simplicity, and stateless horizontal scaling.

Decision Drivers

- **Persona Pain Point**: “As a student stuck on a difficult level, i want to quickly ask for hints without losing my current game state.”
- **Persona Pain Point**: “As a creator, when i publish my level, i want clear, instant confirmation without network hangs.”
- **Technical Constraints**: Must integrate cleanly with Unity’s UnityWebRequest, support standard OAuth 2.0 authentication, and allow easy JSON payload sanitization.

Considered Options

- HTTPS REST API + JSON
- gRPC over HTTP/2
- GraphQL API

Decision Outcome

Chosen Option: HTTPS REST API + JSON

Positive Consequences

- **Direct JSON Serialization**: Native mapping between Unity C# JsonUtility/ Newtonsoft objects and backend JSON data structures for level nodes and state transitions.
- **Simple & stateless**: REST endpoints (POST /levels, GET /levels/{id}, POST /hints) simplify server-side JSON sanitization middleware.
- **Broad Compatibility:** Works seamlessly with third-party OAuth providers (Google/Steam/Auth) and standard CDN distribution models.

Negative Consequences

- Slightly higher payload overhead compared to binary gRPC serialization (negligible for typical level graph JSON sizes under 500KB)